

Программная Инженерия

Создана пользователем [Олег Концебалов](#). Последнее обновление: [только что](#) • Время чтения: 14 мин.

1. Вербальное описание

Тема: Автоматизированная система управления инфраструктурой на bare-metal серверах

Проблема: Современные распределенные системы живут в целом зоопарке инфраструктуры – огромное количество bare-metal серверов (обычные железные сервера), виртуальных машин, различные зоны доступности, дата центры. Каждый сервис необходимо развернуть на каком-либо из этих серверов, поддерживать в рабочем состоянии, настроить мониторинг и алертинг, обновлять до последней версии и не допускать расхождений конфигурации. На практике это чаще всего решается набором разрозненных инструментов и конфигурационных файлов размером в несколько тысяч строк – под каждый дата центр, кластер, систему. Такие конфигурационные файлы очень быстро разрастаются, начинают дублировать друг друга, становятся тяжелыми для сопровождения.

В итоге бизнесу приходится поддерживать целые команды инженеров, которые выполняют все эти задачи руками. Это отнимает у бизнеса огромное количество денег, человеко-часов, а ошибки, которые всегда возникают при ручных действиях затрагивают SLA и клиентов, уменьшая их лояльность и доверие продукту. Нет удобного, единого и легко масштабируемого способа управлять всей этой инфраструктурой как одним целым.

Кажется логичным сделать единое место для описания и управления инфраструктурой на bare-metal серверах – платформу, которая хранит модель систем на уровне приложений и их ресурсов, умеет автоматически отслеживать расхождение фактического и желаемого состояний и устранять это расхождение. Также в целях масштабируемости необходимо предусмотреть возможность добавления в управление платформой произвольных пользовательских приложений и ресурсов.

Назначение системы: автоматизированная система управления инфраструктурой на bare-metal серверах предназначена для централизованного описания, учёта и поддержания согласованного состояния серверов и размещённых на них приложений. Система обеспечивает инженерам единое место для хранения модели инфраструктуры (Inventory) и модели желаемого состояния приложений и конфигураций (Desired State), а также автоматизирует процесс обнаружения и устранения расхождений между фактическим и целевым состоянием. Благодаря этому снижается доля ручных операций при эксплуатации крупных распределённых инфраструктур, уменьшении операционных рисков и повышении стабильности выполнения SLA. Платформа рассчитана на использование в организациях с несколькими датацентрами и зонами доступности, где требуется масштабируемый и единообразный подход к управлению парком серверов.

Суть деятельности: система обеспечивает централизованное управление инфраструктурой bare-metal серверов на основе модели фактического и желаемого состояния. Пользователь задаёт и актуализирует целевые параметры размещения и исполнения приложений, а также сведения о контролируемых серверах. Фактическое состояние хостов и запущенных нагрузок автоматически собирается на стороне серверов и передаётся в управляющий контур. На основе регулярного сопоставления фактических и целевых данных система выявляет расхождения конфигураций и состояния приложений. При обнаружении отклонений формируются корректирующие действия, направленные на восстановление заданного состояния.

Внешние обстоятельства функционирования: система функционирует в условиях высокой неоднородности инфраструктуры: разные типы аппаратных платформ, отличие сетевых и эксплуатационных политик по датацентрам, различные зоны доступности и организационные требования к изменениям. Система должна корректно работать при росте парка серверов, обеспечивая масштабирование подсистем обнаружения расхождений и процесса реконсильации без снижения качества контроля и управляемости изменений.

Основы функционирования:

Архитектура системы построена по модульному принципу и включает два логических слоя:

- Ядро AC (управляющий контур):
 - **Inventory Service** — хранение и предоставление данных о контролируемых серверах и их фактическом состоянии;
 - **DesiredState Service** — хранение и управление целевыми спецификациями приложений и конфигураций;

- **Drift Detector Service** — сравнение фактического и желаемого состояния, формирование событий расхождения;
 - **Reconciler** — приведение инфраструктуры к желаемому состоянию;
 - **Уведомление администратора** — общий механизм состояния по разным этапам восприятия и осознания.
-
- **Scanner** — легкий агент на каждом управляемом хосте, собирающий фактические сведения об ОС, сетевых параметрах и состоянии workload-ов через локальные интерфейсы среды исполнения и ОС. Агент логически относится к платформе, но физически развёрнут на управляемых серверах, обеспечивая достоверность и актуальность инвентарных данных.

2. Структурно функциональный анализ (DFD)

Все диаграммы здесь составлялись с помощью [PlantUML Online Editor](#)

DFD Level 0

► PlantUML Code

```
@startuml
' DFD Level 0 – Контекстная диаграмма
left to right direction
skinparam shadowing false
skinparam rectangleStyle rounded
skinparam ArrowThickness 1.2

actor "Пользователь\n(инженер по сопровождению)" as User
rectangle "AC управления инфраструктурой\nbare-metal серверов" as CP #LightGray
rectangle "Bare-metal сервер" as Host

User --> CP : Обновленная информация о серверах
CP --> User : Информация о серверах
User --> CP : Обновленная информация\nпо желаемом состоянии инфраструктуры
CP --> User : Информация о желаемом\nсостоянии инфраструкутры
CP --> Host : Команды на приведение инфраструктуры\nк желаемому состоянию
Host --> CP : Фактическое состояние\nинфраструктуры на сервере

@enduml
```



DFD Level 1

► PlantUML Code

```
@startuml

skinparam shadowing false
```

```
skinparam rectangleStyle rounded
skinparam ArrowThickness 1.2
```

```
rectangle "Bare-metal сервер" as Host

' Процессы
frame "AC управления инфраструктурой\nbare-metal серверов" {
    rectangle "A1: Inventory Service\n(Управление информацией о серверах)" as P1
    rectangle "A2: DesiredState Service\n(Управление информацией о желаемом\постоянии инфраструкт
    rectangle "A3: Drift Detector\n(Обнаружение расхождений фактического\ни желаемого состояний)"
    rectangle "A4: Reconcile\n(Приведение к желаемому состоянию)" as P4

    ' Хранилища
    database "D1: Inventory Storage\n(PostgreSQL)" as D1
    database "D2: Desired State Storage\n(PostgreSQL)" as D2
}

' --- Потоки от пользователя
User --> P1 : Обновленная информация\по серверах
P1 --> User : Информация\по серверах
User --> P2 : Обновленная информация о\пжелаемом состоянии\пинфраструктуры
P2 --> User : Информация\по желаемом состоянии инфраструктуры

' --- P1 <=> D1
P1 --> D1 : Обновление данных\по серверах
D1 --> P1 : Чтение данных\по серверах

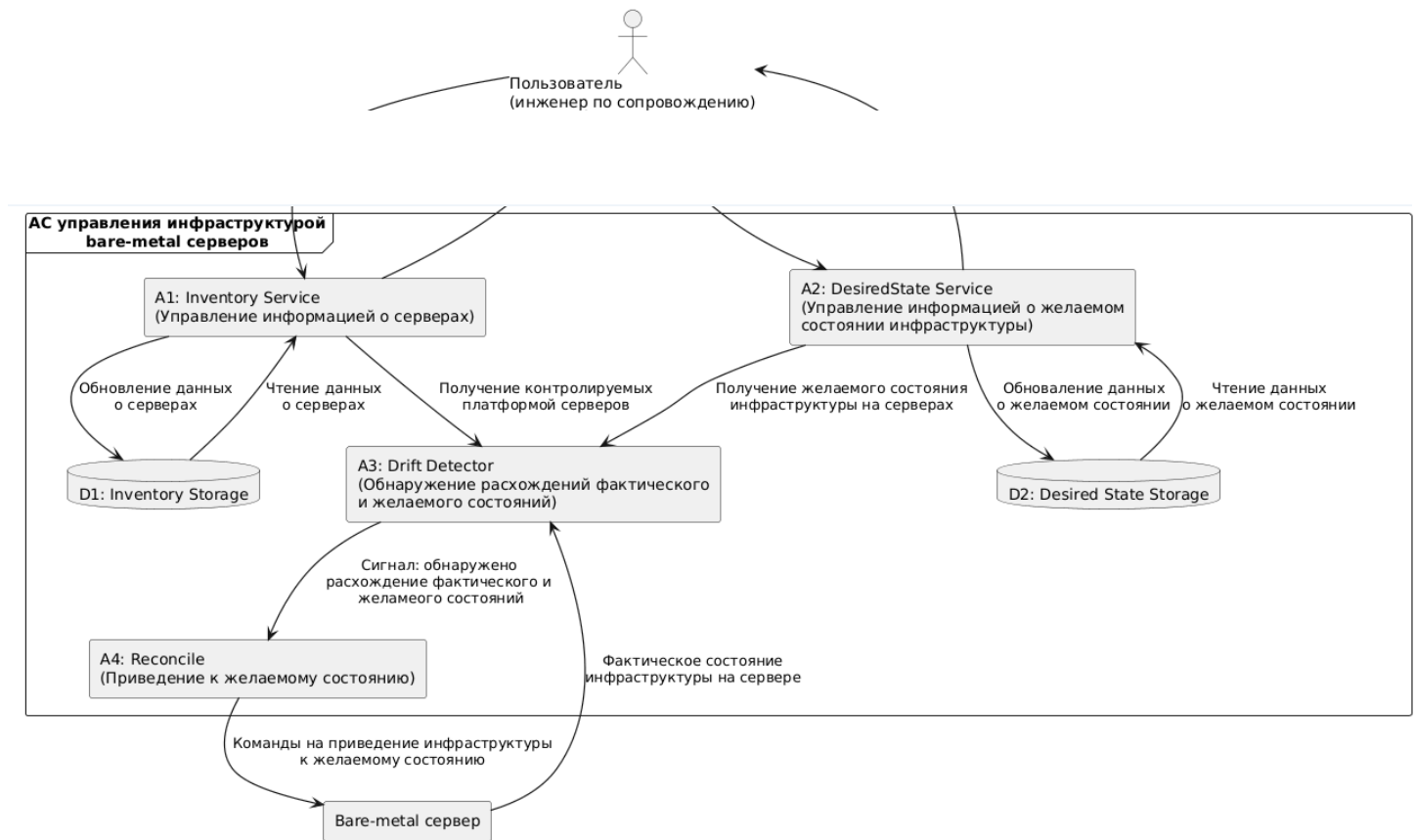
' --- P2 <=> D2
P2 --> D2 : Обновление данных\по желаемом состоянии
D2 --> P2 : Чтение данных\по желаемом состоянии

' --- Чтение данных для детекции дрейфа
P1 --> P3 : Получение контролируемых\пплатформой серверов
P2 --> P3 : Получение желаемого состояния\пинфраструктуры на серверах

' --- Связь детекции и реконсильации
P3 --> P4 : Сигнал: обнаружено\прасхождение фактического и\п желаемого состояний

' --- Взаимодействие с удалённым хостом
Host --> P3 : Фактическое состояние\пинфраструктуры на сервере
P4 --> Host : Команды на приведение инфраструктуры\пк желаемому состоянию

@enduml
```



DFD Level 2

Управление Inventory

› PlantUML Code

```
@startuml
' DFD L2 – A1: Inventory Service (работа с Inventory)

left to right direction

skinparam shadowing false
skinparam rectangleStyle rounded
skinparam ArrowThickness 1.2

' Внешние сущности
actor "Пользователь\n(инженер по сопровождению)" as User
rectangle "A3: Drift Detector\n(Обнаружение расхождений\nфактического и желаемого состояний)" as A3
database "D1: Inventory Storage\n(PostgreSQL)" as D1

' Декомпозиция A1
frame "A1: Inventory Service\n(Управление информацией о серверах)" {

    rectangle "A1.1: Проверка корректности данных" as A11
    rectangle "A1.2: Обновление записей\nInventory" as A12
    rectangle "A1.3: Чтение данных\nInventory" as A13
}

' --- Поток изменения Inventory от пользователя ---
User --> A11 : Обновленная информация\nпо серверах
```

A11 --> A12 : Валидный запрос

A12 --> D1 : Обновление данных\по серверах

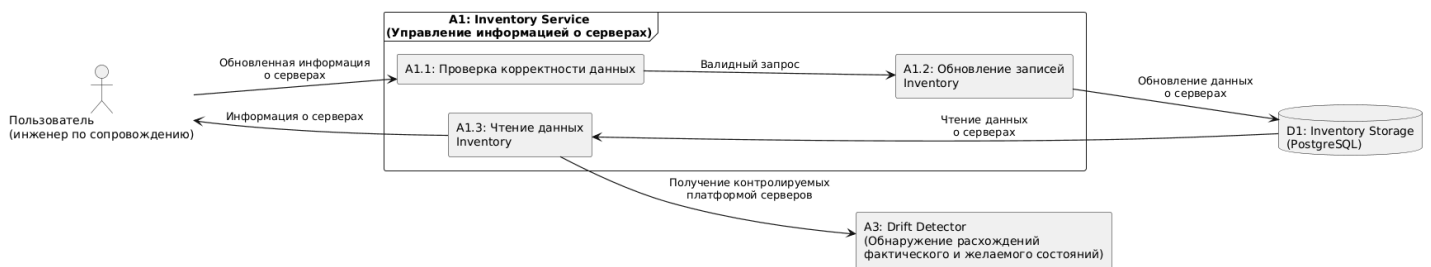
D1 --> A13 : Чтение данных\по серверах

A13 --> User : Информация о серверах

' --- Потоки чтения Inventory для Drift Detector ---

A13 --> Drift : Получение контролируемых\платформой серверов

@enduml



Управление Desired State

> PlantUML Code

```

@startuml
' DFD L2 – A2: DesiredState Service (работа с Desired State)

left to right direction

skinparam shadowing false
skinparam rectangleStyle rounded
skinparam ArrowThickness 1.2

' Внешние сущности
actor "Пользователь\n(инженер по сопровождению)" as User
rectangle "A3: Drift Detector\n(Обнаружение расхождений\nфактического и желаемого состояний)" as A3
database "D2: Desired State Storage\n(PostgreSQL)" as D2

' Декомпозиция A2
frame "A2: DesiredState Service\n(Управление информацией о желаемом\nсостоянии инфраструктуры)"

    rectangle "A2.1: Проверка корректности данных" as A21
    rectangle "A2.2: Обновление записей\nDesired State" as A22
    rectangle "A2.3: Чтение данных\nDesired State" as A23
  }

' --- Потоки изменения Desired State от пользователя ---
User --> A21 : Обновленная информация о\nжелаемом состоянии\nинфраструктуры
A21 --> A22 : Валидный запрос
A22 --> D2 : Обновление данных\по желаемом состоянии
  
```

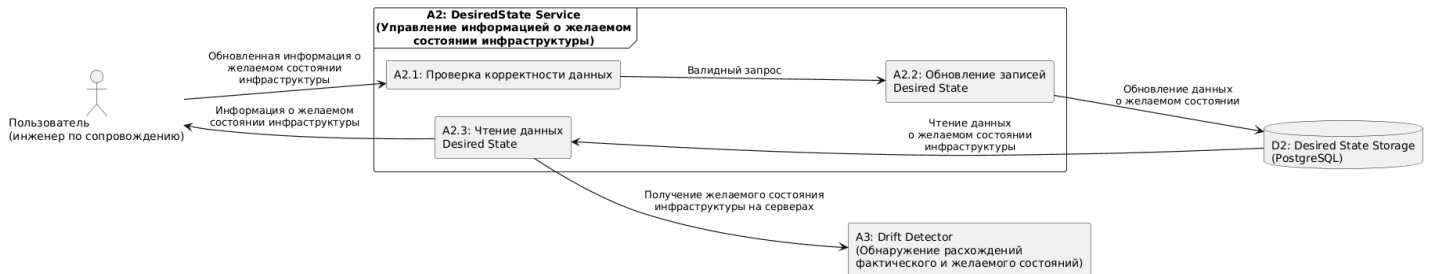
```
' --- Потоки чтения Desired State пользователем ---
```

```
D2 --> A23 : Чтение данных\по желаемом состоянии\инфраструктуры
```

```
A23 --> User: Информация о желаемом\нсостоянии инфраструктуры
```

```
A23 --> Drift : Получение желаемого состояния\инфраструктуры на серверах
```

```
@enduml
```



Обнаружение расхождения конфигурации

► PlantUML Code

```
@startuml
```

```
' DFD L2 – A3: Drift Detector (обнаружение расхождений)
```

```
' left to right direction
```

```
skinparam shadowing false
```

```
skinparam rectangleStyle rounded
```

```
skinparam ArrowThickness 1.2
```

```
' Внешние сущности и хранилища
```

```
rectangle "Bare-metal сервер" as Host
```

```
rectangle "A4: Reconcile\n(Приведение к желаемому\нсостоянию)" as Reconcile
```

```
rectangle "A1: Inventory Service" as A1
```

```
rectangle "A2: DesiredState Service" as A2
```

```
' Декомпозиция A3
```

```
frame "A3: Drift Detector\n(Обнаружение расхождений фактического\ни желаемого состояний)" {
```

```
    rectangle "A3.1: Триггер\nпроверки расхождения\nфактического и желаемого\нсостояний (раз в N ч"
```

```
    rectangle "A3.2: Получение списка\nконтролируемых серверов" as A32
```

```
    rectangle "A3.3: Получение фактического состояния серверов" as A33
```

```
    rectangle "A3.3: Получение данных\nпо желаемом состоянии" as A34
```

```
    rectangle "A3.4: Сравнение состояний\nи формирование события дрейфа" as A35
```

```
}
```

```
' --- Триггер проверки ---
```

```
A31 --> A32 : Старт\nпроверки дрейфа
```

```
' --- Получение списка хостов и фактического состояния ---
```

```
Host --> A33 : Фактическое состояние\nинфраструктуры на сервере
```

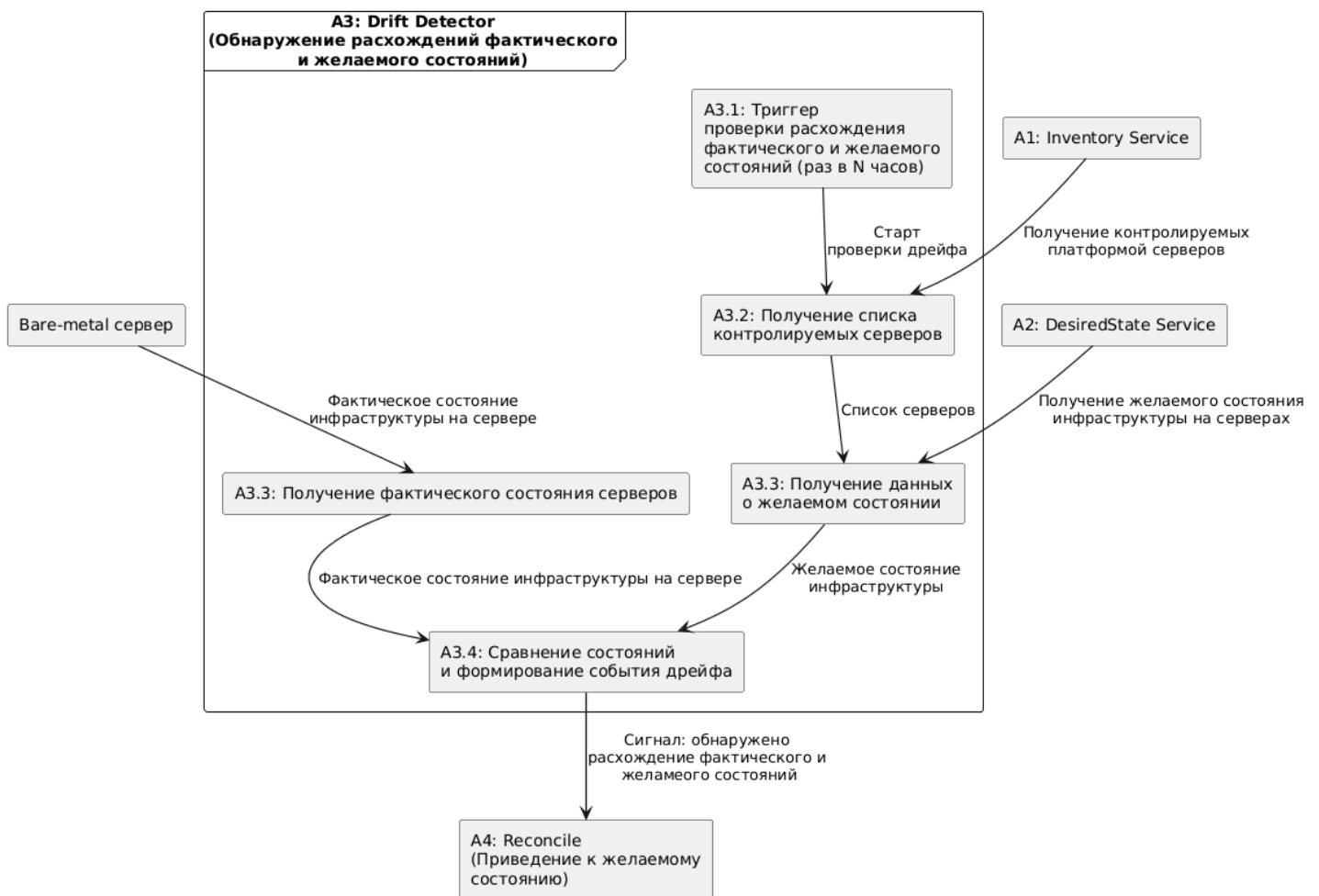
A1 --> A32 : Получение контролируемых\платформой серверов
 A33 --> A35 : Фактическое состояние инфраструктуры на сервере
 A32 --> A34 : Список серверов

A2 --> A34 : Получение желаемого состояния\инфраструктуры на серверах
 A34 --> A35 : Желаемое состояние\инфраструктуры

' --- Сравнение и передача на реконсилляцию ---

A35 --> Reconcile : **Сигнал:** обнаружено\расхождение фактического и\п желаемого состояний

@enduml



Устранение расхождения конфигурации (Реконсилляция)

► PlantUML Code

```

@startuml
' DFD L2 – A4: Reconcile (приведение к желаемому состоянию)

left to right direction

skinparam shadowing false
skinparam rectangleStyle rounded
skinparam ArrowThickness 1.2
  
```

```
' Внешние сущности и хранилище
rectangle "A3: Drift Detector\n(Обнаружение расхождений\nфактического и желаемого состояний)" as
rectangle "Bare-metal сервер" as Host
```

```
frame "A4: Reconcile\n(Приведение к желаемому состоянию)" {

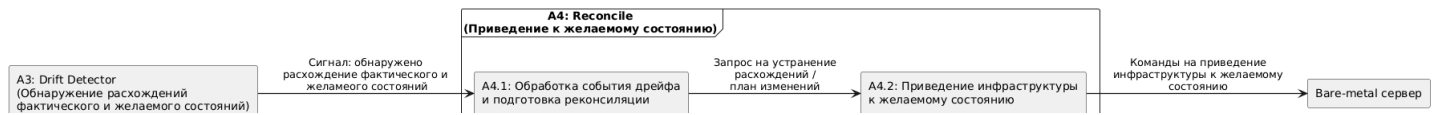
    rectangle "A4.1: Обработка события дрейфа\nи подготовка реконсиляции" as A41
    rectangle "A4.2: Приведение инфраструктуры\nк желаемому состоянию" as A42
}

' --- Вход от Drift Detector ---
Drift --> A41 : Сигнал: обнаружено\nрасхождение фактического и\nжелаемого состояний

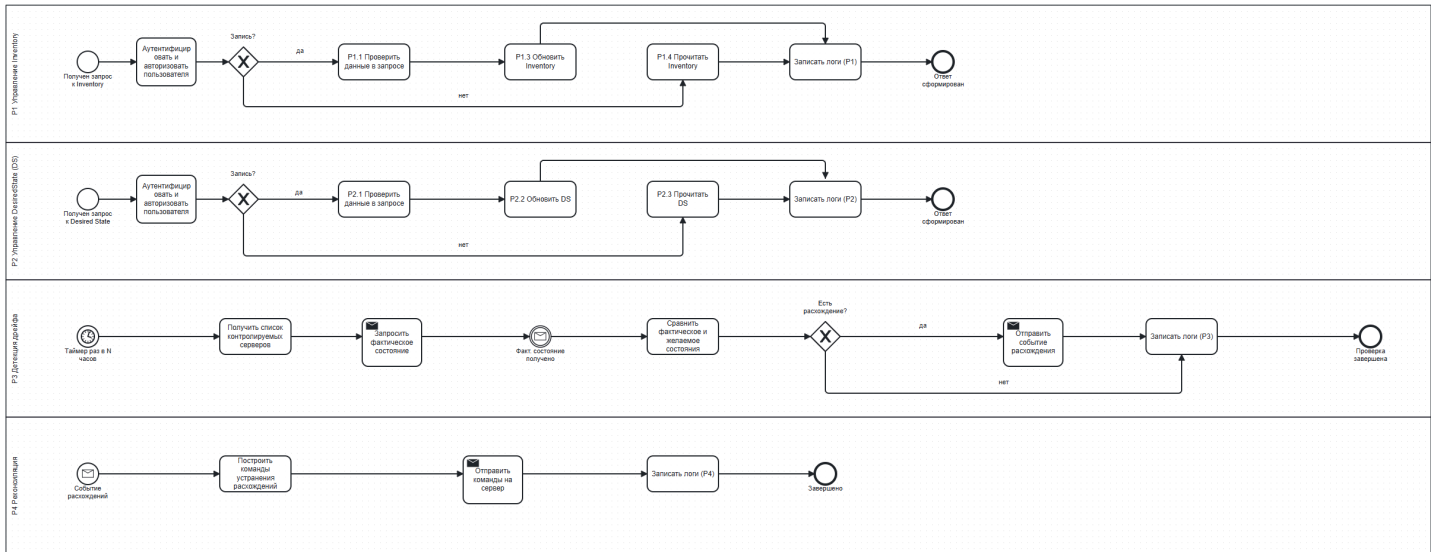
' --- Обработка события и подготовка плана действий ---
A41 --> A42 : Запрос на устранение\nрасхождений /\nплан изменений

' --- Применение изменений на сервере ---
A42 --> Host : Команды на приведение\nинфраструктуры к желаемому\nсостоянию

@enduml
```

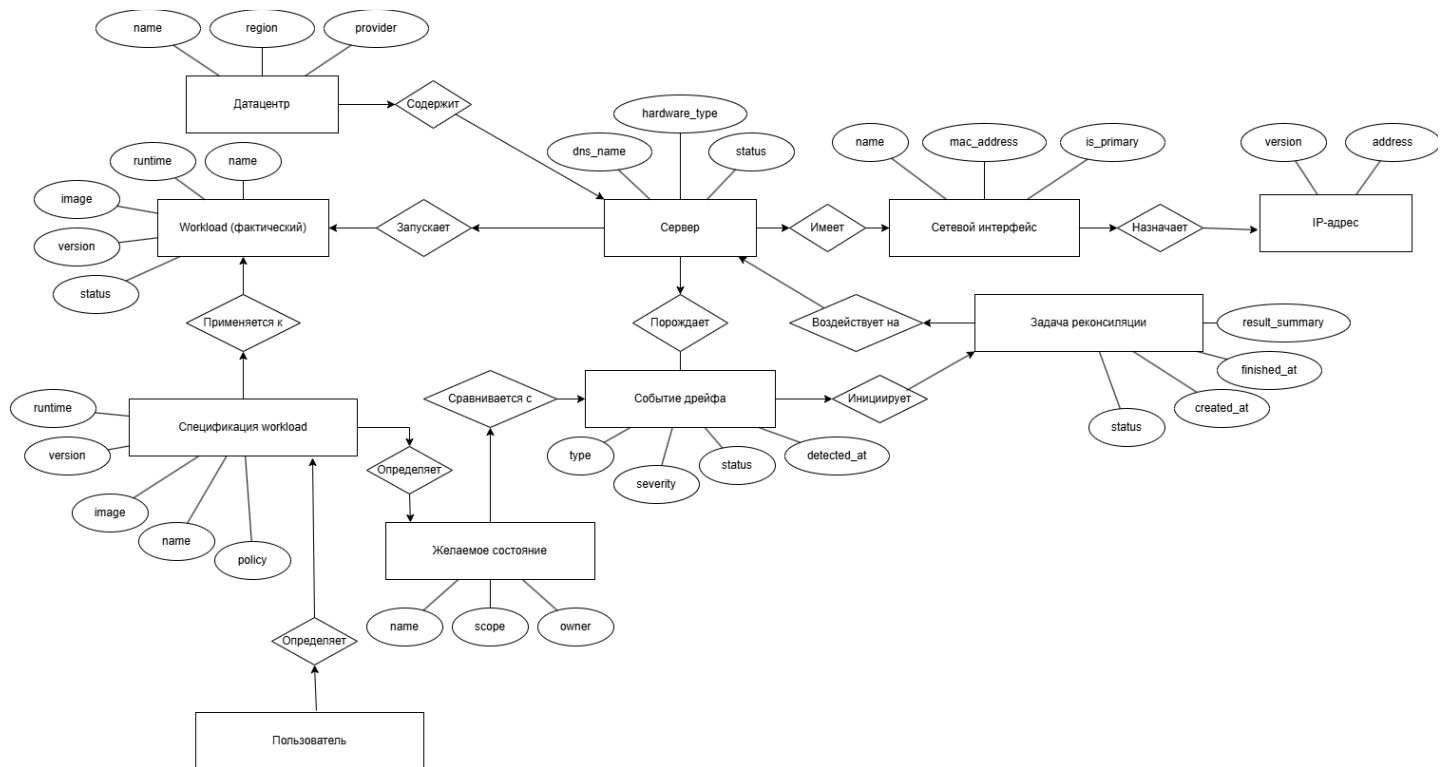



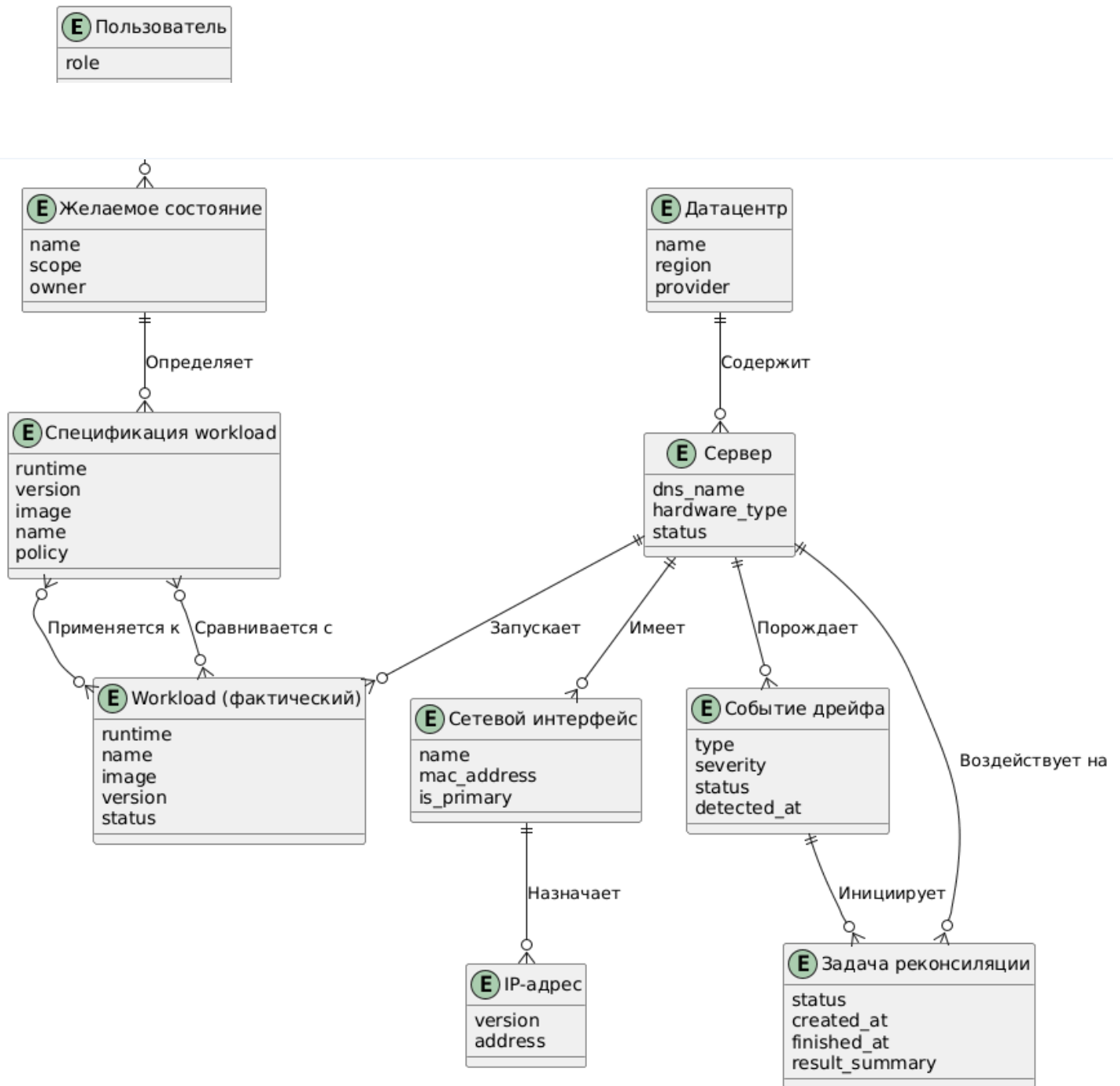
3. Процессные модели (BPMN)



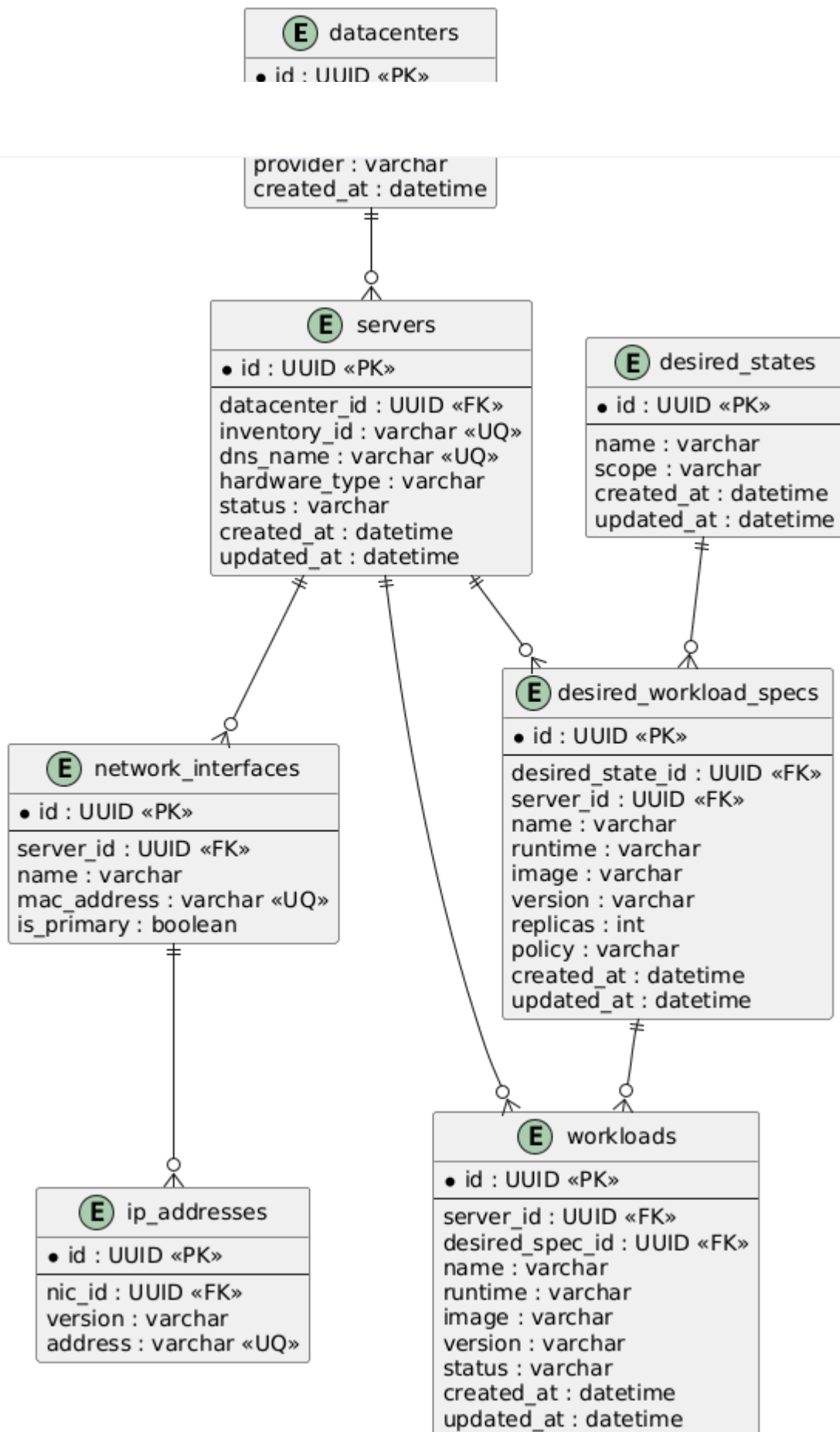
4. ER-диаграмма

Концептуальная

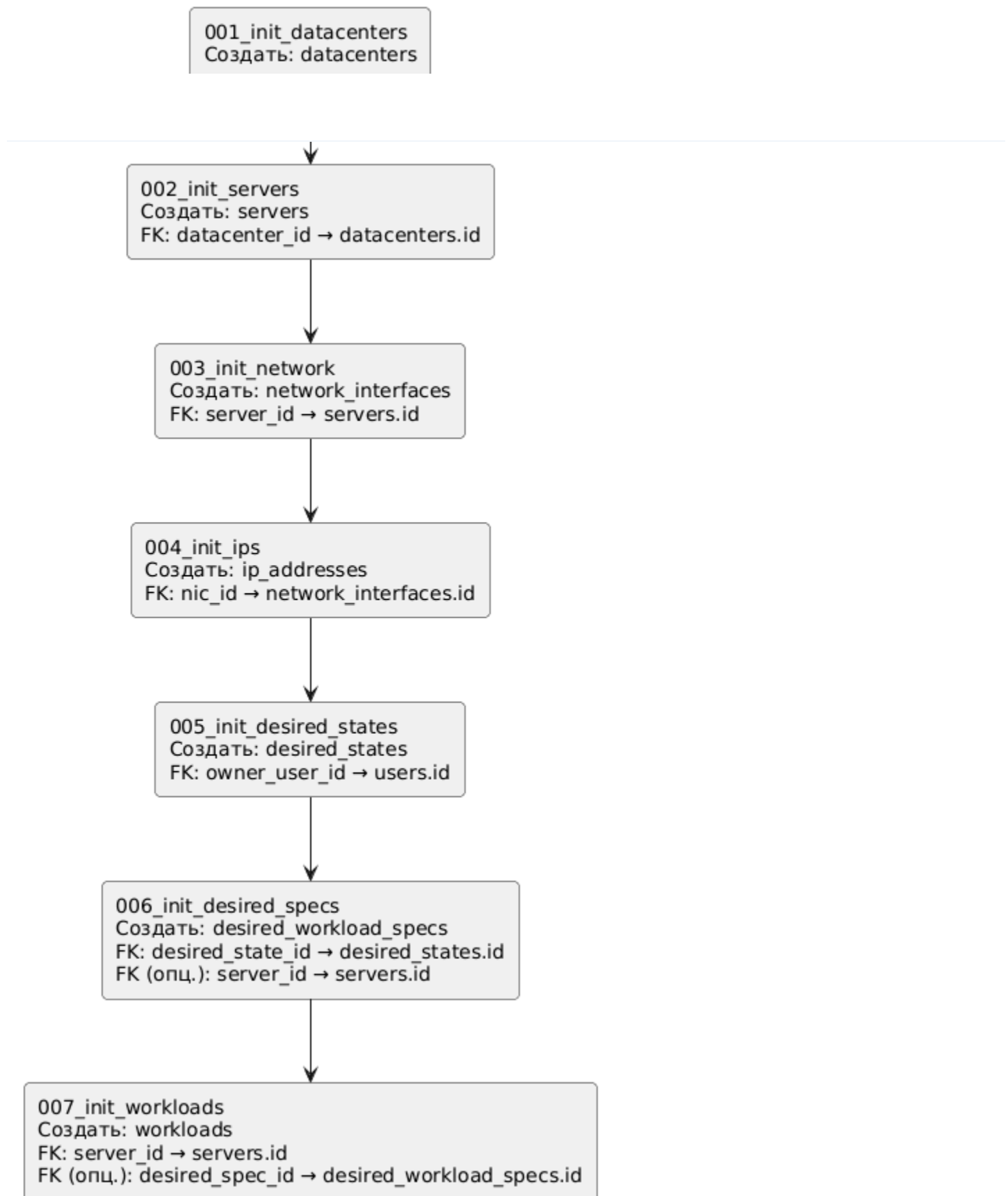




БД-спецификация



Порядок генерации таблиц БД



5. Архитектура

▼ PlantUML Code

```
@startuml
title Архитектура системы
```

```

left to right direction
skinparam shadowing false
skinparam packageStyle rectangle

```

```

}
skinparam note {
  RoundCorner 8
}

actor "Пользователь платформы\n(инженер по сопровождению)" as User

' =====
'  Общая граница системы
' =====

package "АС управления инфраструктурой bare-metal серверов" as CP #EEF5FF {

  ' -----
  '  ПАМКА 1: ControlPlane Core
  ' -----

  package "Ядро AC\n(управляющий контур)" as CORE #EAF3FF {

    rectangle "Inventory Service" as INV #8DBBFF
    rectangle "DesiredState Service" as DSS #8DBBFF
    rectangle "Drift Detector Service" as DRIFT #8DBBFF
    rectangle "Reconciler\n(приводит фактическое состояние\nк желаемому)" as REC #8DBBFF

    package "Хранилище данных (абстрактно)" as DATA #FFFFFF {
      database "Storage\n(контролируемые сервера,\nжелаемое состояние)" as STORE
    }

    INV --> STORE : SQL\n(данные о серверах)
    DSS --> STORE : SQL\n(данные о желаемом состоянии)
  }

  ' -----
  '  ПАМКА 2: Host Runtime Layer
  ' -----

  package "Server Layer\n(исполнение на серверах)" as HOSTL #F7F7F7 {

    rectangle "Scanner\n(лёгкий агент на хосте)" as AGENT #8DBBFF
    rectangle "Bare-metal сервер\nпод управлением платформы" as Hosts #DDDDDD

    AGENT --> Hosts : вызовы CLI\n(локальные состояния,\nконфигурация)
  }
}

' =====
'  Пользовательские взаимодействия
'  (нет единого API)
' =====

User --> INV : HTTP\n(обновление Inventory)

```

User --> DSS : HTTP\n(обновление DesiredState)

' =====

DRIFT --> INV : HTTP\n(запрос контролируемых серверов)

DRIFT --> DSS : HTTP\n(запрос желаемого состояния)

DRIFT --> REC : HTTP\n(события расхождения)

DRIFT --> AGENT : HTTP\n(запрос фактического состояния)

' =====

' Реконсиляция на хост

' =====

REC --> Hosts : SSH

' =====

' Пояснения

' =====

note right of AGENT

Агент логически относится к платформе,
но физически развёрнут на каждом
управляемом хосте.

end note

note right of STORE

Эволюция хранения:

- 1) **Dev:** локальное/встраиваемое хранилище на машине разработчика.
- 2) **MVP/ранняя эксплуатация:** единая централизованная БД для всех сервисов.
- 3) **Масштабирование:**
 - разделение на БД по доменам (Inventory / DesiredState)
 - репликация и кластеризация для повышения сохранности данных и отказоустойчивости
 - read-реплики для аналитики

end note

note left of DRIFT

Эволюция обнаружения расхождений:

- 1) **Dev/ранняя эксплуатация:** сервис в едином экземпляре, обходит все сервера в спокойном режиме.
- 2) **Эксплуатация с ростом количества серверов:** масштабировать сервис на несколько экземпляров для параллельной проверки разных групп серверов

end note

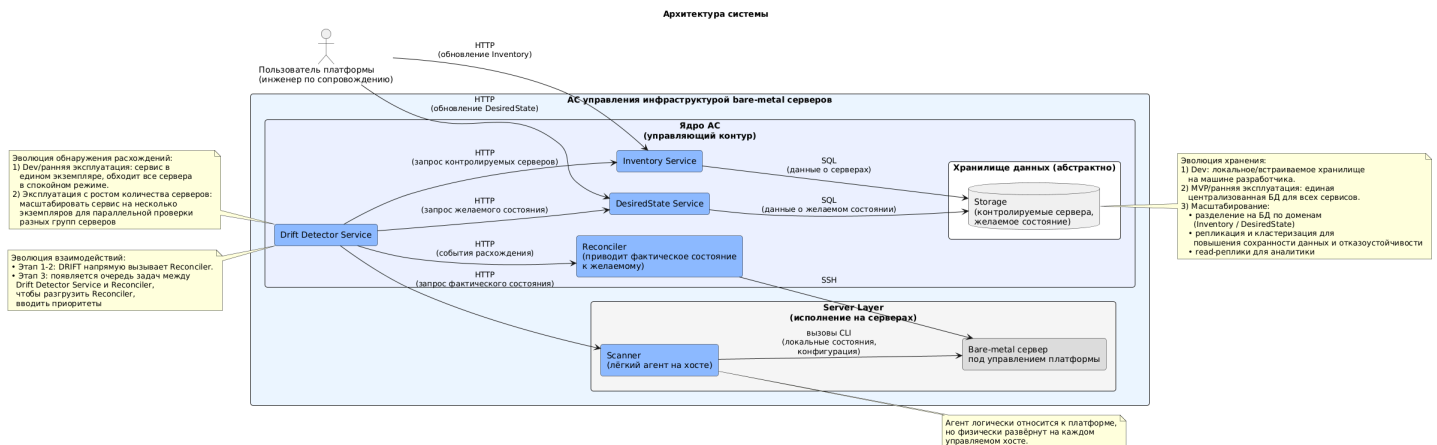
note left of DRIFT

Эволюция взаимодействий:

- **Этап 1-2:** DRIFT напрямую вызывает Reconciler.

- **Этап 3:** появляется очередь задач между Drift Detector Service и Reconciler, чтобы разгрузить Reconciler,

@enduml



Архитектурное описание

1. Объект архитектуры

Объектом архитектурного описания является автоматизированная система управления инфраструктурой bare-metal серверов (далее — **АС**). Система предназначена для централизованного учета управляемых серверов (Inventory), хранения и управления желаемым состоянием целевых нагрузок и конфигураций (Desired State), обнаружения расхождений между фактическим и желаемым состоянием (Drift Detection), а также инициирования и выполнения действий по приведению фактического состояния к целевому (Reconcile).

АС реализует замкнутый управляющий контур, в котором фактическое состояние инфраструктуры периодически собирается с управляемых хостов, сопоставляется с эталонными спецификациями и при наличии расхождений формируются команды корректирующего воздействия.

2. Контекст и границы системы

АС рассматривается как часть эксплуатационной среды датацентра и взаимодействует со следующими внешними сущностями:

- Пользователь платформы (инженер по сопровождению), выполняющий операции управления инвентарем и желаемым состоянием.
- Управляемые bare-metal серверы, на которых размещаются целевые нагрузки.

ВАЖНОЕ архитектурное решение: в текущем варианте отсутствует единый API Gateway; каждый доменный сервис предоставляет собственное API. Пользователь напрямую взаимодействует с **Inventory Service** и **DesiredState Service**, что отражает доменную автономию подсистем и обеспечивает независимость их развития.

В рамки системы входят два логических слоя:

1. Ядро АС (управляющий контур) — сервисы Inventory, DesiredState, Drift Detector, Reconciler и абстрактное хранилище данных.
2. Server Layer (исполнение на серверах) — легковесный агент Scanner и управляемый bare-metal хост. Агент логически относится к платформе, но физически развернут на каждом управляемом сервере.

3. Стейкхолдеры и их интересы

Основные заинтересованные стороны:

Интересы: предсказуемость эксплуатационных изменений, контроль соответствия конфигураций, снижение времени восстановления и ручных операций, удобный доступ к инвентарю и целевым спецификациям.

- **Эксплуатация датацентра**

Интересы: полнота и актуальность инвентарных данных, корректное управление жизненным циклом серверов.

- **Функция информационной безопасности**

Интересы: управляемость доступа, трассируемость действий, минимизация рисков неконтролируемых изменений.

4. Архитектурные интересы

Ключевые архитектурные интересы для системы:

- согласованность фактического и желаемого состояния инфраструктуры;
- надежность и управляемость изменений;
- масштабируемость по числу серверов и скорости появления изменений;
- расширяемость доменной модели (добавление новых видов workload-ов/политик);

5. Точки зрения и архитектурные представления

5.1 Контекстная точка зрения

АС обеспечивает два основных класса операций:

- управление доменными данными: инвентарь серверов и желаемое состояние;
- управление согласованностью: обнаружение и устранение дрейфа.

На контекстном уровне входными данными выступают:

- изменения инвентаря и desired state, инициированные пользователем;
- фактические сведения о конфигурации и состояниях, собранные агентом на хостах.

Выходные данные:

- обновленные сведения о фактическом состоянии;
- результаты реконсильации, подтверждающие приведение инфраструктуры к эталонному виду.

5.2 Функциональная точка зрения (ядро АС)

Inventory Service отвечает за хранение и актуализацию сведений о контролируемых серверах и их характеристиках.

DesiredState Service хранит целевые спецификации конфигураций

Drift Detector Service запрашивает перечень контролируемых серверов и desired state, получает фактическое состояние через агент и выполняет сравнение.

Reconciler инициируется результатами обнаруженного дрейфа и выполняет корректирующее воздействие на сервер через канал управления

Данный набор сервисов реализует функционально завершенный управляющий контур, позволяющий поддерживать инфраструктуру в согласованном состоянии.

5.3 Точка зрения взаимодействий

Сценарий работы системы в штатном режиме:

1. Пользователь обновляет данные инвентаря и/или желаемого состояния через API соответствующих сервисов.
2. Drift Detector периодически запрашивает список серверов (Inventory) и целевые спецификации (DesiredState).
3. Для каждого сервера запрашивается фактическое состояние через Scanner (агент).

4. Выполняется сравнение фактического и желаемого состояния.
5. При обнаружении расхождений формируется событие дрейфа и инициируется процесс реконсиляции.
6. Reconciler применяет корректирующие действия на целевом сервере.
7. Результаты отражаются в хранилище и обновляют фактическое состояние.

I План эволюции

Этап 1. Разработка

Цели: быстрое прототипирование, минимизация инфраструктурных зависимостей.

- использование локального/встраиваемого хранилища;
- развертывание сервисов в единичных экземплярах;
- упрощенные политики сравнения и исправления.

Этап 2. MVP и ранняя эксплуатация

Цели: стабильная работа на ограниченном парке серверов.

- единая централизованная БД для доменных сервисов;
- Drift Detector в одном экземпляре с последовательным обходом серверов;
- прямое взаимодействие Drift Detector → Reconciler без промежуточных буферов;

Этап 3. Масштабирование управляющего контура

Цели: устойчивость при росте числа серверов и интенсивности изменений.

- масштабирование Drift Detector на несколько экземпляров для параллельной проверки групп серверов;
- внедрение очереди задач реконсиляции между Drift Detector и Reconciler для:
 - разгрузки Reconciler,
 - приоритизации задач,
 - сглаживания пиков нагрузки;
- разделение хранилищ по доменам - отдельные контуры данных для Inventory и DesiredState;
- добавление read-реплик для аналитики и отчетности.

6. Спецификация

Drift Detector Service

1. Назначение и роль в системе

Drift Detector Service — модуль управляющего контура автоматизированной платформы управления инфраструктурой bare-metal серверов.

Назначение сервиса — **обнаружение и классификация расхождений** между:

- **фактическим состоянием** управляемых хостов и workload-ов;
- **желаемым состоянием**, заданным пользователями.

Результатом работы является:

1. создание **событий дрейфа (Drift Event)**;
2. при необходимости — постановка **задач на реконсиляцию** напрямую или через очередь (эволюционно).

Сервис не является публичной точкой управления пользователем. Его API предназначено преимущественно для внутренних вызовов платформы и эксплуатационного контроля.

2. Границы ответственности

2.1 Входит в ответственность

- получение фактических снимков состояния хостов (через Scanner/Agent или из Inventory при push-модели);
- сравнение “fact vs desired” по согласованным правилам;
- формирование **drift-классификации** (тип, критичность, затронутые сущности);
- запись Drift Event в хранилище и публикация результата;
- инициирование реконсильации.

2.2 Не входит

- изменение desired state;
- прямое управление хостами;
- выполнение исправляющих действий (это зона Reconciler);
- хранение первичных инвентарных данных (Inventory).

3. Входные данные и источники

3.1 Доменные источники

1. Inventory:

- список целевых серверов;
- метаданные хоста (dns, hardware, dc, сетевые параметры);
- при push-архитектуре — актуальный ActualSnapshot в базе.

2. Desired State:

- DesiredWorkloadSpec (runtime/image/version/replicas/policy/placement);
- область применения (host / группа / dc / global).

3. Scanner/Agent:

- фактические состояния workload-ов и хоста;
- локальные признаки деградации, версии образов, параметры запуска.

4. Выходные данные

- **Drift Event:**
 - тип дрейфа (конфигурационный, версионный, топологический, runtime-ошибка и т.п.);
 - критичность (low/medium/high/critical);
 - время обнаружения;
 - ссылка на хост/объект;
 - краткая “diff-сводка” (что отличается);
 - статус обработки.
- **Reconcile Task** (в зависимости от архитектурного этапа):
 - либо прямой вызов Reconciler

5. Функциональные требования

5.1 Основные функции

1. **Периодическое сканирование** парка серверов согласно расписанию.
2. **Точечная проверка** конкретного хоста по требованию.
3. **Сравнение состояния** по согласованной схеме/версии спецификаций.
4. **Классификация дрейфа** и принятие решения об автозапуске реконсильации.
5. **Запись событий** в хранилище и обеспечение идемпотентности.

6. Алгоритм обнаружения дрейфа (логическая модель)

6.1 Шаги проверки

1. Получить **целевое множество спецификации**:
 - спецификации, привязанные к хосту;
 - спецификации по группам/политикам размещения;
 - глобальные политики (если задано).
2. Получить **фактическое состояние**:
 - prefer: свежий ActualSnapshot (если push в Inventory);
 - fallback: запрос к агенту GET /snapshot.
3. Нормализовать данные в единую модель сравнения:
 - приведение версий/тегов образов к каноническому виду;
 - согласование имен/меток workload-ов.
4. Сравнить:
 - наличие/отсутствие workload-ов;
 - runtime;
 - image и version;
 - число экземпляров/реплик (если применимо);
 - policy-параметры.
5. Сформировать:
 - тип дрейфа;
 - список различий (diff);

7. Интерфейсы сервиса

7.1 Внутреннее HTTP API (пример)

Базовый префикс: /internal/v1

1. Запуск плановой проверки:
 - POST /drift/scan
 - body: { "scope": "all|datacenter|host|group", "id": "..." }
 - ответ: { "job_id": "...", "accepted": true }
1. Проверка конкретного хоста:
 - POST /drift/scan/host/{hostId}
1. Получить статус job:
 - GET /drift/jobs/{jobId}
1. Получить события дрейфа:
 - GET /drift/events?host_id=&severity=&status=&since=
1. Подтвердить обработку/закрытие события:
 - POST /drift/events/{eventId}/ack
 - POST /drift/events/{eventId}/resolve

Если нужен минимализм: достаточно 1–2 эндпоинтов на запуск и просмотр событий.

7.2 Интеграция с Inventory

Варианты реализации:

- GET /inventory/hosts
- GET /inventory/hosts/{id}
- GET /inventory/hosts/{id}/actual-snapshot (если push-модель)

7.3 Интеграция с Desired State

- GET /desired/specs?scope=host|group|dc|global&id=

Два режима:

Pull-режим (базовый):

- Drift → Agent:
 - GET /agent/v1/snapshot
 - ответ: нормализованный ActualSnapshot

8. Нефункциональные требования

8.1 Производительность

- сервис должен обеспечивать проверку N хостов в пределах заданного окна времени;
- оптимизация за счёт:
 - батч-чтения данных;
 - параллельных воркеров;
 - кэширования desired-спецификаций на короткий TTL.

8.2 Масштабирование

- горизонтальное масштабирование через:
 - шардирование по группам хостов/datacenters;
 - распределение job на несколько экземпляров.

8.3 Надёжность

- идиоматичная генерация DriftEvent (не плодить дубликаты);
- корректная работа при недоступности агента:
 - событие типа AGENT_UNREACHABLE,
 - статус деградации, без ложного реконсайла.

8.4 Безопасность

- внутренние вызовы — mTLS или сервисные JWT;
- ограничение ролей на эндпоинты запуска сканирования;
- аудит операций обнаружения и запуска исправлений.

8.5 Наблюдаемость

Обязательные метрики:

- drift_scans_total
- drift_events_total{severity,type}
- drift_scan_duration_seconds
- agent_request_failures_total
- reconcile_tasks_created_total

Логи:

- корреляционный request_id/job_id;

Трассировка:

- цепочка Drift → Agent → Inventory/Desired → Reconciler.

9. Предполагаемая реализация на Golang

9.1 Технологии/библиотеки

-
- Хранилище:
 - pgx + sqlc (PostgreSQL),
 - modernnc.org/sqlite или mattn/go-sqlite3 для dev.
 - Конфиги: viper или чистый env-parser.
 - Логи: zap / zerolog.
 - Метрики: prometheus/client_golang.
 - Трассировка: OpenTelemetry SDK.

10. Тестирование Drift Detector

10.1 Unit

- сравнение моделей;
- классификация severity;
- идемпотентность Upsert.

10.2 Интеграционные

- Inventory + Desired + Storage;
- эмуляция агента.

10.3 Сквозные

Сценарий:

1. задать desired state;
2. изменить факт на тестовом хосте;
3. получить Drift Event;
4. проверить создание задачи реконсильации;
5. убедиться в исчезновении дрейфа после исправления.

Нет меток